Informe Técnico — SARA: Asistente Virtual de Contratación Pública

SERCOP — Coordinación de Tecnología de la Información y Comunicaciones Fecha: Abril 2026 | Versión: 1.0

---

1. Resumen Ejecutivo

SARA (Sistema de Asesoría y Respuesta Automatizada) es un asistente virtual de WhatsApp que responde preguntas sobre normativa de contratación pública ecuatoriana, citando artículos de la LOSNCP, el Reglamento General y resoluciones vigentes del SERCOP.

El sistema fue instalado y configurado en el servidor de producción del SERCOP. Durante las pruebas se identificó una limitación crítica de hardware (ausencia de GPU) que elevaba los tiempos de respuesta a 50-100 segundos. Esta limitación fue resuelta integrando **Groq API** como proveedor del modelo de lenguaje, reduciendo los tiempos a **1-2 segundos** manteniendo toda la base de conocimiento normativo en la infraestructura local del SERCOP.

---

2. Infraestructura del Servidor de Producción

Componente	Detalle
Servidor	Linux RHEL (Red Hat Enterprise Linux)
Procesador	Intel Xeon Gold 6342 — 10 cores activos
RAM	31 GB
GPU	**Ninguna**
Python	3.9
Ubicación	Infraestructura interna SERCOP

2.1 Servicios corriendo en producción

Servicio	Puerto	Descripción
SARA (FastAPI + Uvicorn)	8000	Servidor principal del bot
Ollama	11434	Motor LLM local (CPU)
PostgreSQL	5432	Base de datos conversaciones + RAG
nomic-embed-text	—	Modelo de embeddings (vía Ollama)

---

3. Arquitectura del Sistema

```
Usuario WhatsApp
      ↓
Meta Cloud API → POST /webhook
      ↓
FastAPI (main.py)
  ├─ Responde 200 OK inmediato a Meta (evita timeout de 5s)
  └─ Procesa en background:
        ↓
      brain.py — Pipeline de respuesta
        ├─ 1. Clasificar mensaje (saludo vs. consulta)
        ├─ 2. Pre-tool execution (Python detecta intención por keyword)
        │     montos PIE, plazos, RUP, fecha, tipo contratación
        ├─ 3. RAG — pgvector + tsvector (solo si no hay tool suficiente)
        │     3,059 chunks de 17 documentos normativos indexados
        └─ 4. LLM — genera respuesta con contexto inyectado
              ↓
           Groq API (Llama 3.3 70B) o Ollama local (fallback)
              ↓
           Respuesta → Meta API → WhatsApp usuario
```

3.1 Base de Conocimiento Normativa (RAG)

Documento	Tipo	Chunks
Reglamento General LOSNCP — octubre 2025	Reglamento	1,132
Normativa Secundaria de Contratación Pública 2025	Reglamento	1,125
LOSNCP — RO 140 07-X-2025	Ley	113
Manual SOCE — Subasta Inversa Electrónica	Manual	108
RE-SERCOP-2026-0001, 0002 y otras resoluciones	Resoluciones	456
Otros documentos (manuales, glosario, pliegos)	Varios	125
TOTAL		**3,059 chunks**

3.2 Tools disponibles (datos estructurados, 100% precisos)

Tool	Activación	Dato que entrega
`obtener_montos_pie`	Pregunta por montos/umbrales	Valores exactos 2025/2026 por tipo de proceso
`obtener_plazos`	Pregunta por plazos/días	Plazos de 7 tipos: SIE, menor cuantía, licitación, impugnación, contrato, garantías
`info_rup`	Pregunta sobre proveedores/RUP	Requisitos, costo, tiempo, renovación

Tool	Activación	Dato que entrega
`obtener_fecha_hora_ecuador`	Pregunta por fecha/hora	Fecha y hora actual Ecuador (UTC-5)
`recomendar_tipo_contratacion`	Pregunta qué proceso usar	Proceso recomendado con normativa

---

4. Limitación Crítica: Ausencia de GPU

4.1 Impacto medido en pruebas

El motor LLM (Ollama) corre íntegramente en CPU. La limitante no es la cantidad de núcleos sino el **ancho de banda de memoria RAM** del servidor, que determina a qué velocidad se pueden leer los pesos del modelo durante la inferencia.

Modelo	Tamaño	Tiempo de respuesta (CPU puro)
gemma3:1b	1.3 GB	~25-30s (pero alucina frecuentemente)
qwen2.5:3b	2.2 GB	~44-80s
gemma3:4b	4.4 GB	~80-100s
gemma4:e4b	9.6 GB	~80-120s
gemma4:26b	17 GB	**NO VIABLE** — 3-5 min + inestabilidad

4.2 Por qué no se puede resolver solo con más núcleos

La inferencia de LLMs es una operación **memory-bound** (ligada a la memoria), no compute-bound. Agregar más núcleos de CPU no reduce el tiempo de forma significativa. La única solución de hardware es una GPU con alta velocidad de memoria (GDDR6X o HBM).

4.3 Comparativa con GPU

Hardware	Tiempo por respuesta	Modelo recomendado
CPU Intel Xeon (actual)	44-100 segundos	qwen2.5:3b
GPU NVIDIA RTX 4090 (24GB VRAM)	2-5 segundos	gemma4:26b
GPU NVIDIA A100 (80GB VRAM)	1-2 segundos	gemma4:26b

---

5. Solución Implementada: Groq API

Para resolver la limitación de GPU sin incurrir en el costo de hardware, se integró **Groq API** como proveedor del LLM. Groq utiliza hardware especializado (LPU — Language Processing Unit) que procesa LLMs a velocidades muy superiores a GPU convencionales.

5.1 Resultados obtenidos

Mensaje	Antes (CPU local)	Después (Groq)
"Hola"	~80s	**1.1s**
"Qué es el RUP"	~97s (respuesta incorrecta)	**1.7s** (respuesta completa y correcta)
Consulta normativa compleja	~100s	**2-3s**

5.2 Modelo utilizado

Llama 3.3 70B Versatile — modelo de Meta con 70 mil millones de parámetros, excelente comprensión del español y razonamiento jurídico.

5.3 Flujo de datos con Groq

```
[Infraestructura SERCOP]              [Groq — USA]
       ↓                                     ↓
  RAG local (pgvector)              Llama 3.3 70B
  Tools locales (Python)    →→→→→   Recibe: system prompt +
  Embeddings locales                        contexto normativo +
                                            pregunta del usuario
                                    ←←←←←  Devuelve: respuesta en texto
```

> **Nota de privacidad:** El texto de las consultas de los usuarios y los fragmentos de normativa recuperados por el RAG se envían a los servidores de Groq (ubicados en Estados Unidos) para generar la respuesta. Esto debe ser validado con el área legal/compliance del SERCOP antes del despliegue oficial, considerando la normativa de datos institucionales.

---

6. Costos

6.1 Plan gratuito de Groq (situación actual)

Límite	Valor	Impacto
Requests por minuto	30 RPM	Hasta 30 usuarios simultáneos por minuto
Tokens por minuto	6,000 TPM	~2-3 conversaciones simultáneas
Requests por día	14,400	~480 conversaciones completas/día
Costo mensual	**$0**	Gratuito indefinidamente dentro de límites

6.2 Plan de pago (si el uso supera el gratuito)

Precio Groq para **Llama 3.3 70B**: $0.59 / millón tokens entrada · $0.79 / millón tokens salida

Cada mensaje consume aproximadamente:

- **Entrada:** ~2,100 tokens (system prompt + RAG + historial + pregunta)
- **Salida:** ~400 tokens (respuesta de SARA)

Escenario	Mensajes/día	Costo input/mes	Costo output/mes	**Total/mes**
Piloto inicial	100	$1.86	$0.95	**~$3**
Uso normal	500	$9.30	$4.74	**~$14**
Alto volumen	2,000	$37	$18.96	**~$56**
Máximo estimado SERCOP	5,000	$93	$47	**~$140**

6.3 Comparativa de opciones

Opción	Costo mensual	Tiempo respuesta	Privacidad datos	Requiere
Groq (actual)	$0-14	1-2s	⚠️ Datos salen al exterior	API key gratuita
GPU RTX 4090	~$0 operativo + $2,500 hardware	2-5s	✅ 100% local	Adquisición GPU
GPU cloud (AWS/GCP)	$200-800/mes	1-3s	⚠️ Datos en nube	Proveedor cloud
CPU local (actual sin Groq)	$0	44-100s	✅ 100% local	Nada adicional

---

7. Configuración Actual del Sistema

```
# Proveedor LLM
LLM_PROVIDER=groq
GROQ_MODEL=llama-3.3-70b-versatile

# LLM local (fallback si Groq no responde)
OLLAMA_MODEL=qwen2.5:3b
OLLAMA_NUM_CTX=8192
OLLAMA_MAX_TOKENS=400

# RAG
TOP_K_CHUNKS=2
RERANKER_ENABLED=false
HISTORIAL_LIMITE=4
```

---

8. Riesgos y Consideraciones

Riesgo	Probabilidad	Impacto	Mitigación
Groq supera límite gratuito (14,400 req/día)	Baja en piloto	Medio	Activar plan pago (~$14/mes)

Riesgo	Probabilidad	Impacto	Mitigación
Groq no disponible (caída del servicio)	Baja	Alto	Fallback automático a Ollama local
Datos normativos viajan a servidores Groq	Certeza	A validar legalmente	Validar con área legal SERCOP
Respuesta lenta si Groq cae y usa Ollama local	Baja	Medio	Monitoreo de uptime
Meta reintenta webhook si no recibe 200 en 5s	Mitigado	Bajo	BackgroundTasks implementado

---

9. Recomendaciones

Corto plazo (piloto)

- ✅ Usar Groq plan gratuito para el piloto — sin costo, 1-2s de respuesta
- ✅ Validar con área legal/compliance el envío de datos a Groq
- ✅ Monitorear uso diario para anticipar si se necesita plan de pago

Mediano plazo (producción oficial)

- Evaluar adquisición de **GPU NVIDIA RTX 4090** (~$2,500) para operación 100% local, sin dependencia externa y sin limitación de privacidad
- Con GPU: `gemma4:26b` local, 2-5s de respuesta, datos 100% en SERCOP

Largo plazo

- Completar base de conocimiento: 5 manuales SOCE pendientes + COA
- Evaluación RAGAS (calidad del RAG): meta faithfulness > 0.85
- Deploy en servidor RHEL de producción con gemma4:26b + GPU

---

10. Estado del Sistema al Cierre de Pruebas

Componente	Estado
Webhook WhatsApp (Meta Cloud API)	✅ Configurado
Base de conocimiento RAG (3,059 chunks)	✅ Indexada en PostgreSQL
Pre-tool execution (5 tools)	✅ Funcionando
Groq API integrada	✅ Activa — 1-2s respuesta
Fallback Ollama local	✅ Configurado (qwen2.5:3b)
Compatibilidad Python 3.9 (servidor RHEL)	✅ Corregida
BackgroundTasks (200 OK inmediato a Meta)	✅ Activo

Componente	Estado
Reranker	⏸️ Desactivado (RERANKER_ENABLED=false)
Documentos faltantes (5 manuales SOCE + COA)	🔧 Pendiente
Evaluación RAGAS	⏳ Pendiente

Elaborado por: Coordinación TIC — SERCOP Ecuador Generado con asistencia de Claude Code (Anthropic)